REPORT ON SPECIFICATION-DRIVEN DEVELOPMENT (SDD)

1. Introduction

The software development industry is undergoing a significant transformation with the introduction of Artificial Intelligence (AI) into engineering workflows. AI-powered coding assistants and autonomous development agents can now generate application code, create APIs, identify bugs, write test cases, and assist with system architecture. These capabilities have considerably reduced the effort required to implement software solutions. However, generating code faster does not necessarily mean developing software more accurately.

One of the major challenges in AI-assisted software engineering is ensuring that generated software satisfies the actual requirements of users, stakeholders, and organizations. AI systems interpret natural language instructions and generate solutions based on the information provided. When these instructions are incomplete, ambiguous, or inconsistent, the resulting implementations may differ from the intended functionality.

Traditional prompt-based development often involves asking an AI assistant to generate code through a series of conversational instructions. Although this approach is effective for small tasks and experimentation, it becomes difficult to maintain consistency when applications involve multiple components, developers, business rules, and technical constraints.

Specification-Driven Development (SDD) addresses these challenges by establishing a structured specification as the foundation of software development. Rather than relying primarily on individual prompts, SDD requires developers to define what the system should accomplish, how it should behave, and which conditions must be satisfied before implementation begins.

The specification acts as a shared reference for developers, AI agents, testers, architects, and stakeholders throughout the software development lifecycle.

This approach emphasizes an important engineering principle: the specification defines the intended system, while the implementation demonstrates how that system is constructed.

GitHub's Spec Kit and Microsoft's guidance on AI-native engineering describe this approach as a way of preserving engineering intent across planning, code generation, testing, and validation.

2. What is Specification-Driven Development (SDD)?

Specification-Driven Development is a software engineering methodology in which development begins with a clearly defined, structured, and verifiable specification rather than directly with source code or implementation instructions.

A specification is a formal or semi-formal description of the functional requirements, expected behavior, constraints, interfaces, and success conditions of a software system. It communicates what the software is expected to achieve and establishes the boundaries within which implementation must occur.

In SDD, specifications are treated as active engineering artifacts rather than documents that are written once and forgotten. They guide system design, implementation, automated testing, quality assurance, and future modifications.

The central idea is that developers and AI agents should not independently interpret vague requirements each time a feature is implemented. Instead, every engineering activity should refer to an agreed-upon specification that captures the original intent of the system.

For example, consider developing an e-commerce application. A simple instruction such as "Create a shopping website with product search and cart functionality" provides a general idea of the application but leaves many questions unanswered. It does not explain how product searches should behave, whether users need authentication, how inventory should be managed, what happens during payment failures, or which security conditions must be satisfied.

Under Specification-Driven Development, these requirements are documented explicitly. The specification might define that users can search products by name, add available products to a cart, review the total price, authenticate before checkout, and receive an order confirmation only after successful payment verification.

By documenting these expectations before coding begins, SDD reduces uncertainty and provides a clear foundation for implementation.

Thoughtworks describes SDD as an approach that makes structured specifications the primary reference for AI-assisted software generation, validation, and maintenance.

Thoughtworks United Kingdom+1

2.1 Core principles of SDD

The fundamental principles of Specification-Driven Development include requirement clarity, traceability, consistency, verifiability, and continuous refinement.

Requirement clarity ensures that software behavior is described explicitly before implementation. Traceability establishes relationships between requirements, generated code, and test cases. Consistency ensures that different components follow the same rules and interfaces. Verifiability makes it possible to determine objectively whether the implementation satisfies its requirements.

Continuous refinement is equally important because software requirements are rarely permanent. As business needs evolve, specifications must be updated and reviewed so that implementations remain aligned with the latest approved requirements.

SDD therefore does not mean that every technical decision must be finalized before development. It means that important requirements, decisions, assumptions, and changes should be made explicit and available for review.

3. Traditional Development vs Prompt-Driven Development vs Specification-Driven Development

Traditional software engineering generally begins with requirement gathering, followed by system design, coding, testing, and deployment. Requirements may be captured through software requirement specification documents, user stories, diagrams, and technical design documents.

Prompt-driven development introduces AI assistants into this process. Developers describe the required functionality through natural language prompts, and AI models generate code in response. Development may continue through additional prompts requesting changes or corrections.

SDD extends these approaches by maintaining an explicit specification as the authoritative reference for implementation and validation.

Aspect	**Traditional Development**	**Prompt-Driven Development**	**Specification-Driven Development**
Starting point	Requirements and design documents	Natural language prompts	Structured, reviewed specification
Primary reference	Requirements, design and code	Conversation and generated outputs	Approved specification
AI involvement	Optional	Central to code generation	Guided by the specification

Requirement consistency	Depends on documentation	Can decline across prompts	Supported by shared requirements
Testing	Often planned separately	Frequently generated on request	Derived from acceptance criteria
Change management	Documentation and code updates	Repeated prompt modifications	Controlled specification updates
Traceability	Depends on project practices	Often limited	Explicitly emphasized

The principal difference is not simply the amount of documentation produced. It is the role that documentation plays in controlling engineering decisions. In SDD, specifications influence what gets built, what changes are permitted, and how completion is evaluated.

4. Why the Specification, Not the Prompt, Becomes the Source of Truth

One of the most important concepts in Specification-Driven Development is the distinction between a prompt and a specification.

A prompt is an instruction supplied to an AI model to request a particular response or action. It may ask the AI to generate code, modify an existing function, explain an error, or implement a feature. Prompts are valuable tools for communicating tasks, but they are not necessarily complete descriptions of system requirements.

A specification, by contrast, is a structured and persistent record of what the system must do. It defines expected behavior, boundaries, constraints, interfaces, and measurable conditions of success.

In AI-assisted engineering, this distinction becomes essential because multiple AI interactions may occur throughout the development process.

4.1 Prompts are temporary instructions

Prompts are often created for immediate tasks and may reflect only a portion of the complete application requirements. For instance, a developer may initially ask an AI tool to create a login page and later request a password reset function.

If each request is treated independently, the AI may produce implementations that use inconsistent authentication methods, conflicting validation rules, or incompatible user data structures.

Additionally, prompts may evolve as developers experiment with different solutions. The most recent prompt is not necessarily the most accurate description of the product's intended behavior.

A specification addresses this problem by maintaining a consistent reference that is independent of individual conversations.

4.2 Specifications preserve engineering intent

Engineering intent refers to the underlying objectives and requirements that a software system is supposed to satisfy.

A prompt generally describes an immediate action, such as creating a particular component. A specification explains the purpose of that component, the behavior expected from it, and how it interacts with the larger system.

Consider the following prompt:

> Create a login page using React with email and password fields.

This instruction does not define authentication security requirements, error handling, session management, or access restrictions.

A specification would instead state that the application must authenticate registered users, validate input, avoid exposing sensitive information, and prevent unauthorized access to protected pages.

The developer or AI agent can then determine the appropriate implementation approach while preserving these essential requirements.

4.3 Specifications reduce ambiguity

Natural language requirements frequently contain ambiguous expressions such as "fast," "secure," "user-friendly," or "highly scalable."

For example, a prompt stating that an application should respond quickly does not define an acceptable response time.

A measurable specification might require a particular API endpoint to return a response within 500 milliseconds at the 95th percentile under an agreed test workload.

This transforms a subjective requirement into a verifiable condition.

Similarly, an instruction to provide secure authentication can be expanded into specific requirements for password hashing, transport encryption, access control, session handling, and protection against common attacks.

By resolving ambiguity before implementation, specifications reduce the number of assumptions that AI tools or developers must make.

4.4 Specifications support consistency across multiple AI agents

Modern software projects may use different AI agents for frontend development, backend implementation, database design, testing, and documentation.

Without shared requirements, these agents may generate incompatible components. A frontend agent might expect a field called `productName`, while a backend agent returns `name`. Similarly, one component may represent product prices as floating-point values while another expects integer values in the smallest currency unit.

A shared specification establishes data contracts, naming conventions, API behavior, and validation requirements that all agents must follow.

This improves integration and reduces the need for repeated corrections.

4.5 Specifications enable objective validation

A prompt is primarily an instruction to produce an output. A specification also establishes the conditions against which that output can be evaluated.

For example, suppose an AI agent generates a product search feature. The specification may state that searches must be case-insensitive, support partial product names, and return an empty list when no products match.

These rules can be converted into automated test cases.

The resulting code is considered acceptable only when it satisfies the defined conditions, rather than merely appearing functional.

4.6 Specifications improve change management

Software systems frequently evolve after their initial implementation. New business rules, security requirements, or customer expectations may require modifications.

In an unstructured prompt-driven workflow, the reason behind earlier implementation decisions may be difficult to recover.

SDD addresses this by allowing specifications to evolve alongside the software. When a requirement changes, the specification is reviewed and updated, and affected components and tests can be identified.

For example, if an e-commerce application originally allows checkout without authentication but later requires registered users, the updated specification can identify the authentication requirement, affected user flows, API dependencies, and acceptance tests.

This establishes a traceable record of why the system changed.

4.7 Specifications strengthen accountability

A specification also provides a shared agreement among product managers, architects, developers, testers, and other stakeholders.

Stakeholders can review requirements before development, identify missing functionality, and confirm expected behavior. Developers can use the specification to make implementation decisions, and testers can independently verify compliance.

The specification is therefore an authority for intended behavior, but it does not replace source code, tests, or operational evidence. An implementation must still be reviewed and verified, and the specification itself must be kept accurate.

Microsoft's discussion of SDD emphasizes this alignment of requirements, architecture, implementation, and validation, rather than relying on disconnected AI prompts.

5. Anatomy of a Good Specification

A high-quality software specification must be sufficiently detailed to guide implementation without unnecessarily restricting valid technical solutions.

The most important components are intent, scope, constraints, interfaces, acceptance criteria, and test cases. Each component serves a distinct purpose in ensuring that the system is understandable, implementable, and verifiable.

Core structure of a software specification



5.1 Intent

Intent describes the fundamental purpose of the software system or feature. It explains the problem being solved, the users who will benefit, and the expected business or operational outcomes.

A good specification begins with intent because understanding why a feature is required helps developers make appropriate technical decisions.

For example, the intent of an online shopping cart is not merely to store product identifiers in a list. Its purpose is to allow customers to select products, review intended purchases, understand costs, and proceed toward checkout.

Clearly expressing intent prevents developers and AI agents from focusing exclusively on visible functionality while overlooking the underlying user requirement.

An effective intent statement should identify the problem, the intended users, the expected outcome, and the reason the feature is valuable.

Example intent statement:

> The purpose of the shopping cart module is to enable customers to select available products, modify quantities, review the estimated total amount, and proceed to a reliable checkout process without losing their selections during normal navigation.

This statement communicates the desired user experience without prescribing detailed implementation techniques.

5.2 Scope

Scope defines the boundaries of a software project or feature. It specifies which capabilities must be developed and which capabilities are deliberately excluded.

Scope is particularly important in AI-assisted development because AI agents may generate additional functionality that appears useful but was never requested.

For example, when instructed to build a shopping application, an AI model might add product recommendations, payment integrations, user analytics, and inventory management even when the project only requires a basic product listing and shopping cart.

Although these additions may appear beneficial, they introduce unnecessary complexity, development effort, and testing requirements.

A well-defined specification separates in-scope functionality from out-of-scope functionality.

For a basic e-commerce application, the scope may include displaying products, searching by name, adding products to a cart, reviewing cart contents, and completing a simulated checkout.

Features such as real-time delivery tracking, personalized recommendations, multi-vendor marketplaces, and live payment processing may be explicitly excluded from the first version.

This separation prevents uncontrolled feature expansion, commonly known as scope creep.

Scope should also identify assumptions and dependencies that could affect implementation. For instance, the application may assume that product information is already available through a backend API.

5.3 Constraints

Constraints describe the mandatory conditions and limitations under which the software must operate.

They restrict implementation choices to ensure that the system meets organizational, technical, security, performance, and operational requirements.

Constraints differ from functional requirements. A functional requirement describes what the system does, while a constraint limits how the system can be designed or operated.

For instance, an application may require React for frontend development, FastAPI for backend services, and PostgreSQL for data storage. These decisions can be documented as technology constraints when they are mandatory.

Other constraints may concern security, response time, compatibility, infrastructure, legal compliance, and budget.

Common categories include:

- Technical constraints: Required frameworks, programming languages, databases, or infrastructure.
- Performance constraints: Response-time targets, throughput, and resource limits.
- Security constraints: Authentication, authorization, encryption, and secure handling of information.
- Operational constraints: Deployment environments, availability, monitoring, and backup requirements.
- Compatibility constraints: Supported browsers, devices, API versions, and operating systems.

A good constraint must be clear enough to validate.

For example, "The system must be fast" is ambiguous, whereas "The product search API must achieve a 95th-percentile response time below 500 ms under 100 concurrent virtual users in the defined test environment" is measurable.

Not every constraint needs to be determined during the earliest requirements phase. SDD commonly separates the initial specification of what and why from the technical plan describing how, while recording mandatory technology decisions wherever they apply.

5.4 Interfaces

Interfaces describe how users, software components, services, and external systems interact with the application.

Well-defined interfaces are essential because modern applications generally consist of several independently developed components.

A typical web application may include a React frontend, a FastAPI backend, a PostgreSQL database, and external authentication or payment services. Each component must communicate using agreed data formats, endpoints, and behavioral expectations.

Interface specifications typically describe API endpoints, HTTP methods, request and response schemas, data types, authentication requirements, error formats, and the behavior of user-facing interactions.

The specification should define the meaning and allowed values of every field. It should also state how the API behaves when no products exist, the database is unavailable, or a request is invalid.

For a production system, an interface contract may additionally include versioning rules, pagination, authorization, rate limits, and standardized error responses.

This reduces integration problems and allows frontend and backend components to be developed independently while preserving compatibility.

5.5 Acceptance criteria

Acceptance criteria are explicit conditions that must be satisfied before a feature is considered complete and acceptable.

They translate high-level requirements into observable system behavior.

Acceptance criteria should be specific, measurable, testable, and directly associated with functional or non-functional requirements.

One commonly used method is the Given–When–Then format.

- Given: The initial state or precondition.
- When: The action or event being performed.
- Then: The expected system response.

For example:

> Given that a customer is viewing an available product, when the customer clicks "Add to Cart," then the product should appear in the cart with a quantity of one and the correct price.

Acceptance criteria are especially valuable for AI-generated software because they provide objective expectations against which implementations can be evaluated.

For a shopping cart feature, acceptance criteria might require that products can be added successfully, quantities can be updated only within available stock limits, cart totals are calculated correctly, and empty carts cannot proceed to order placement.

The difference between acceptance criteria and general requirements is the level of verification. A requirement may state that users should be able to manage a cart. The acceptance criteria define the exact conditions under which that capability is considered successfully implemented.

5.6 Test cases

Test cases provide the procedures, inputs, conditions, and expected outputs used to verify that software satisfies its requirements and acceptance criteria.

While acceptance criteria describe what successful behavior looks like, test cases describe how that behavior will be checked.

A comprehensive specification should contain or reference tests for normal operations, boundary conditions, invalid inputs, security-sensitive actions, and failure scenarios.

For example, testing an Add to Cart function may involve verifying that the requested item exists, its quantity is valid, sufficient stock is available, and the calculated total is correct.

Test cases should also cover negative situations, such as attempting to add an unavailable product, requesting a negative quantity, or submitting invalid product identifiers.

In Specification-Driven Development, AI agents can use these test descriptions to generate unit tests, integration tests, and end-to-end tests. However, AI-generated tests still require review to ensure that they verify intended requirements rather than simply reproducing assumptions made by the implementation.

A traceable relationship should exist between requirements, acceptance criteria, and corresponding tests.

6. Practical Example: Specification for an E-Commerce Shopping Cart

To understand how the six components work together, consider a simplified shopping cart module for an online product store.

6.1 Feature overview

Feature name: Shopping Cart Management

Specification ID: SDD-CART-001

Version: 1.0

Objective: Allow customers to add products to a cart, update quantities, remove products, view the cart total, and proceed to checkout.

6.2 Intent

The shopping cart feature should provide customers with a straightforward way to organize products before purchasing them. It should maintain accurate quantities and totals while preventing customers from requesting unavailable stock.

6.3 Scope

The feature includes adding products, updating product quantities, removing individual items, displaying selected products, calculating totals, and continuing to a checkout page.

The initial version excludes discount coupons, shipping estimation, multi-currency calculations, and real payment gateway processing.

6.4 Constraints

The frontend must use React, the backend must expose REST APIs through FastAPI, and product information must be retrieved from PostgreSQL.

Product prices must be represented internally as integer amounts in the smallest currency unit to avoid floating-point rounding errors. Quantities must be positive integers and cannot exceed available stock.

The application must not trust prices supplied by the browser. Prices must be retrieved and validated on the server before an order is confirmed.

6.5 Interfaces

The proposed API contract includes:

Method	Endpoint	Function
GET	`/cart`	Retrieve cart contents
POST	`/cart/items`	Add a product
PATCH	`/cart/items/{product_id}`	Update quantity

DELETE	`/cart/items/{product_id}`	Remove a product
GET	`/cart/total`	Retrieve calculated cart total

An example request to add a product is:

```
None
{
  "product_id": 101,
  "quantity": 2
}
```

A successful response could be:

```
None
{
  "message": "Item added to cart",
  "product_id": 101,
  "quantity": 2,
  "cart_total_minor": 159800,
  "currency": "INR"
}
```

In this example, `cart_total_minor` represents paise, so the cart total is ₹1,598.00.

The interface contract should also state that invalid quantities produce an HTTP 422 response, unavailable product identifiers produce HTTP 404, and insufficient stock produces HTTP 409. These are proposed rules for this example, not universal HTTP requirements for all cart systems.

6.6 Acceptance criteria

ID	Acceptance criterion

AC-01	An available product can be added to the cart
AC-02	A product cannot be added if the requested quantity exceeds stock
AC-03	Cart quantities can be modified using positive integers
AC-04	Removing a product updates the cart total
AC-05	The cart total equals the sum of server-validated item prices multiplied by their quantities
AC-06	An empty cart cannot proceed to order submission

6.7 Test cases

Test ID	Test scenario	Expected result	Maps to
TC-01	Add product 101 with quantity 1	Product appears in cart	AC-01
TC-02	Add quantity 20 when stock is 15	Request rejected with HTTP 409	AC-02

TC-03	Update quantity from 1 to 3	Quantity and total update	AC-03, AC-05
TC-04	Remove the only cart item	Cart becomes empty and total becomes zero	AC-04
TC-05	Add two products with different prices	Correct total is returned	AC-05
TC-06	Submit an order with an empty cart	Order submission is rejected	AC-06
TC-07	Provide a negative quantity	Validation rejects the request	AC-03

This example demonstrates how the specification connects business intent with engineering implementation and test verification.

Instead of asking an AI coding assistant to simply generate a cart component, developers can provide the complete specification and instruct the AI to implement only the documented functionality.

The generated implementation can then be evaluated against the acceptance criteria and test cases, reducing ambiguity and helping reviewers identify missing functionality.

7. Specification-Driven Development Workflow

SDD follows a systematic development process in which the specification guides each major engineering activity.

Typical SDD lifecycle

1. Define intent and requirements

Identify the problem, users, and expected outcomes

2. Create and review specification

Document scope, constraints, interfaces, and acceptance criteria

3. Plan architecture and tasks

Design components, dependencies, and implementation steps

4. Implement with AI assistance

Generate and refine code using the approved specification

5. Validate against specification

Execute tests, review behavior, and resolve gaps

Review and refine when requirements change

The workflow begins with requirement analysis, during which developers and stakeholders identify the problem, users, and desired outcomes.

Next, the team creates a specification that captures functional requirements and important non-functional conditions. Ambiguities and conflicts are reviewed before implementation.

During planning, developers or AI agents translate the specification into architecture decisions, modules, interfaces, dependencies, and executable tasks.

Implementation then proceeds using the approved requirements. AI tools may generate application components, API endpoints, data models, tests, and documentation, while developers review the resulting artifacts.

Finally, the implementation is validated against acceptance criteria, test cases, security requirements, and integration contracts. Any differences between expected and actual behavior are investigated and corrected.

GitHub Spec Kit supports this approach through structured stages such as Specify, Plan, Tasks, Implement, and Converge, with additional clarification and quality-checking activities available when needed.

8. Benefits of Specification-Driven Development in AI-Assisted Engineering

Specification-Driven Development offers several benefits when software teams use generative AI and autonomous coding agents.

8.1 Improved requirement clarity

By documenting intended behavior before implementation, SDD encourages developers to identify missing information and resolve conflicting assumptions. This improves communication between technical and non-technical stakeholders.

8.2 Reduced implementation rework

When requirements are incomplete, developers may repeatedly modify generated code to correct misunderstood functionality. Well-defined specifications can reduce this rework by identifying critical expectations earlier.

8.3 More reliable AI-generated code

AI tools work with clearer task boundaries when requirements, interfaces, and constraints are explicit. This does not guarantee correct code, but it provides stronger context for generation and a basis for detecting mistakes.

8.4 Better test coverage

Acceptance criteria and test cases can be created before or alongside implementation. This encourages teams to test not only successful operations but also error handling, boundary conditions, and invalid inputs.

8.5 Easier collaboration

A shared specification helps frontend developers, backend developers, testers, architects, and AI agents operate from consistent assumptions. It is particularly useful in projects that involve parallel engineering activities.

8.6 Improved maintainability and traceability

Specifications preserve the rationale behind important software requirements. Future developers can understand why a feature behaves in a particular way, and changes can be traced to associated requirements and tests.

9. Challenges and Limitations of SDD

Although Specification-Driven Development provides a structured foundation for software engineering, it also introduces practical challenges.

The first challenge is the effort required to prepare high-quality specifications. Developers must invest time in identifying requirements, clarifying edge cases, and defining measurable acceptance conditions. For small experiments or temporary scripts, extensive specification activities may not be justified.

A second challenge is specification quality. An incomplete or incorrect specification can still produce an incorrect implementation even when AI agents follow it precisely. The presence of structured documentation does not automatically guarantee that the underlying requirements are accurate.

Another concern is specification drift, which occurs when implementation changes are made without updating the approved specification. Over time, the documented behavior may no longer match the actual application.

SDD also requires discipline in change management. Teams need a clear process for approving requirement modifications, updating dependent interfaces, and reviewing affected tests.

Finally, AI-assisted implementation still requires human oversight. Specifications may not anticipate every security vulnerability, performance limitation, integration failure, or unexpected user behavior. Generated code must therefore be reviewed, tested, and monitored independently.

These trade-offs are also highlighted in industry discussions of SDD, which emphasize specification quality and governance as important conditions for successful adoption.

Thoughtworks United Kingdom

10. Best Practices for Writing Effective Specifications

An effective specification should communicate requirements clearly enough for both human engineers and AI agents to interpret them consistently.

Teams should use precise language and avoid undefined terms such as "simple," "efficient," or "secure." Where possible, requirements should be expressed through measurable outcomes and observable behavior.

Each significant requirement should have a unique identifier so that it can be referenced by design decisions, implementation tasks, and test cases. This makes requirement traceability easier throughout the software lifecycle.

Specifications should distinguish mandatory requirements from optional preferences. They should also clearly identify assumptions, external dependencies, excluded functionality, and unresolved questions.

Interfaces should be treated as explicit contracts, particularly when multiple development teams or AI agents are working on independent components.

Acceptance criteria should focus on externally observable behavior, and test cases should include positive, negative, boundary, and failure scenarios.

Specifications should be maintained using version control where practical. This allows teams to review changes, compare revisions, and recover earlier decisions.

Most importantly, developers should treat the specification as a living engineering artifact. It must evolve when requirements change, while remaining consistent with the architecture, implementation, and tests.

11. Conclusion

Specification-Driven Development represents an important shift in how software can be designed, implemented, and validated in AI-assisted engineering environments.

As generative AI makes code generation increasingly accessible, the main engineering challenge is no longer limited to writing code quickly. It also involves defining the correct system, preserving intended behavior, coordinating multiple components, and verifying that implementations satisfy actual user requirements.

SDD addresses these needs by placing a structured specification at the center of the development lifecycle. Instead of depending on isolated prompts, teams establish a shared reference that communicates system intent, scope, constraints, interfaces, acceptance criteria, and test cases.

Each element contributes to software quality. Intent explains why the system is needed, scope establishes its boundaries, constraints define mandatory conditions, interfaces regulate interactions, acceptance criteria establish success conditions, and test cases provide evidence that those conditions have been satisfied.

In this approach, prompts remain useful for directing AI tools, but they are subordinate to the approved specification. AI agents assist with planning, generating, refining, and testing software while the specification preserves the engineering intent behind those activities.

Ultimately, Specification-Driven Development shifts the emphasis from generating code that appears to work toward engineering software that demonstrably satisfies clearly defined requirements. Its value lies not in eliminating human decisions, but in making those decisions explicit, reviewable, traceable, and verifiable.